1. Contest

1.1 Snippet

```
#include <bits/stdc++.h>
using namespace std;
typedef int64_t ll;
#define int ll
#define all(v) v.begin(),v.end()
#define pii pair<int,int>
template <typename T> using v =
std::vector<T>;
const ll inf = ll(4e18) + 5;
const char nl = '\n';
signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    return 0;
}
```

1.2 Pragmas

```
#pragma GCC optimize("O3")
#include <bits/allocator.h>
#pragma GCC target("avx2")
#include <bits/stdc++.h>

#pragma GCC optimize("O3,unroll-loops")
// #pragma GCC
target("avx2,bmi,bmi2,lzcnt,popcnt")
```

1.3 Bitset

```
bitset<size> b;
b[i]        // 0/1
b.set(i)    // b[i]=1
b.reset(i)  // b[i]=0
b.flip(i)
b.count()   // #ones
b.any(), b.none(), b.all()
b1 &= b2
b1 |= b2
b1 ^= b2
b << k , b >> k
bitset<N> b(string)   // MSB at index 0
bitset<N> b(ull x)    // from number
b.to_ullong()  // N <= 64
b.to_string()
```

1.4 Mint

```
const int MOD = 998244353;
struct Mint {
    int v;
    Mint(ll x = 0) { v = x % MOD; if (v < 0) v
+= MOD; }

    explicit operator int() const { return v; }
    Mint& operator+=(const Mint& o) { if ((v
+= o.v) >= MOD) v -= MOD; return *this; }
    Mint& operator-=(const Mint& o) { if ((v
-= o.v) < 0) v += MOD; return *this; }
    Mint& operator*=(const Mint& o) { v =
1LL * v * o.v % MOD; return *this; }
    Mint& operator/=(const Mint& o) { return
*this *= o.inv(); }

    friend Mint operator+(Mint a, const
Mint& b) { return a += b; }
    friend Mint operator-(Mint a, const Mint&
b) { return a -= b; }
    friend Mint operator*(Mint a, const
Mint& b) { return a *= b; }
    friend Mint operator/(Mint a, const Mint&
b) { return a /= b; }

    Mint operator-() const { return Mint(-v); }
    Mint& operator++() { return *this += 1; }
    Mint& operator--() { return *this -= 1; }

    friend bool operator==(const Mint& a,
const Mint& b) { return a.v == b.v; }
    friend bool operator!=(const Mint& a,
const Mint& b) { return !(a == b); }

    Mint inv() const { return pow(MOD - 2); }
    Mint pow(ll p) const {
        Mint res = 1, a = *this;
```

```
        while (p) { if (p & 1) res *= a; a *= a; p >>= 1; }
        return res;
    }

    friend ostream& operator<<(ostream& os, const Mint& a) { return os << a.v; }
    friend istream& operator>>(istream& is, Mint& a) { ll x; is >> x; a = Mint(x); return is; }
};
```

1.5 Stress

```
g++ sol.cpp -o sol
g++ brute.cpp -o brute
g++ gen.cpp -o gen

for ((i=1; ; i++)); do
    ./gen $i > in
    ./sol < in > out
    ./brute < in > ok
    diff -w out ok || break
    echo "Passed $i"
done

chmod +x run.sh
cat in
cat out  # Your wrong output
cat ok   # The correct brute force output
```

1.6 Other Mint

```
typedef unsigned long long ull;
ull modmul(ull a, ull b, ull M) {
    ll ret = a * b - M * ull(1.L / M * a * b);
    return ret + M * (ret < 0) - M * (ret >= (ll)M);
}
ull modpow(ull b, ull e, ull mod) {
    ull ans = 1;
    for (; e; b = modmul(b, b, mod), e /= 2)
        if (e & 1)
            ans = modmul(ans, b, mod);
    return ans;
}
ll modLog(ll a, ll b, ll m) {
    ll n = (ll)sqrt(m) + 1, e = 1, f = 1, j = 1;
    unordered_map<ll, ll> A;
    while (j <= n && (e = f = e * a % m) != b % m)
        A[e * b % m] = j++;
    if (e == b % m)
        return j;
    if (__gcd(m, e) == __gcd(m, b))
        rep(i, 2, n + 2)
            if (A.count(e = e * f % m))
                return n * i - A[e];
    return -1;
}
typedef unsigned long long ull;
ull sumsq(ull to) {
    return to / 2 * ((to - 1) | 1);
}
/// ^ written in a weird way to deal with overflows correctly
ull divsum(ull to, ull c, ull k, ull m) {
    ull res = k / m * sumsq(to) + c / m * to;
    k %= m;
    c %= m;
    if (!k)
        return res;
    ull to2 = (to * k + c) / m;
    return res + (to - 1) * to2 - divsum(to2, m - 1 - c, m, k);
}
ll modsum(ull to, ll c, ll k, ll m) {
    c = ((c % m) + m) % m;
    k = ((k % m) + m) % m;
    const ll mod = 1000000007, LIM = 200000;
    ll* inv = new ll[LIM] - 1;
    inv[1] = 1;
    rep(i, 2, LIM)
        inv[i] = mod - (mod / i) * inv[mod % i] % mod;
    return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}
```

```
ll sqrt(ll a, ll p) {
   a %= p;
   if (a < 0) a += p;
   if (a == 0) return 0;
   assert(modpow(a, (p - 1) / 2, p) == 1); //
else no solution
   if (p % 4 == 3)
      return modpow(a, (p + 1) / 4, p);
   ll s = p - 1, n = 2;
   int r = 0, m;
   while (s % 2 == 0)
      ++r, s /= 2;
   /// find a non-square mod p
   while (modpow(n, (p - 1) / 2, p) != p - 1)
      ++n;
   ll x = modpow(a, (s + 1) / 2, p);
   ll b = modpow(a, s, p);
   ll g = modpow(n, s, p);
   for (;; r = m) {
      ll t = b;
      for (m = 0; m < r && t != 1; ++m)
         t = t * t % p;
      if (m == 0)
         return x;
      ll gs = modpow(g, 1LL << (r - m - 1),
p);
      g = gs * gs % p;
      x = x * gs % p;
      b = b * g % p;
   }
}
bool isPrime(ull n) {
   if (n < 2 || n % 6 % 4 != 1)
      return (n | 1) == 3;
   ull A[] = {2, 325, 9375, 28178, 450775,
9780504, 1795265022},
      s = __builtin_ctzll(n - 1),
      d = n >> s;
   for (ull a : A) { // ^ count trailing zeroes
      ull p = modpow(a % n, d, n), i = s;
      while (p != 1 && p != n - 1 && a % n
&& i--)
         p = modmul(p, p, n);
      if (p != n - 1 && i != s)
         return 0;
   }
   return 1;
}
```

1.7 RNG

```
mt19937_64 rng((unsigned
int)chrono::steady_clock::now().time_since_
epoch().count());

uniform_int_distribution<long long> dist(1,
100);
uniform_real_distribution<double> dist(0.0,
1.0);
normal_distribution<double> dist(50.0,
10.0);
bernoulli_distribution dist(0.75); <- bool
exponential_distribution<double> dist(0.5);
dist(rng); rng();
shuffle(nums.begin(), nums.end(), rng);
```

1.8 Decimal Precision

```
cout << "Fixed(4): " << fixed <<
setprecision(4) << pi << endl;
```

How to compare decimals
For a <= b it should be a <= b + EPS
For a < b it should be a + EPS < b

1.9 Builtin GCC Stuff

- __builtin_clz(x): the number of zeros at the beginning of the bit representation.
- __builtin_ctz(x): the number of zeros at the end of the bit representation.
- __builtin_popcount(x): the number of ones in the bit representation.
- __builtin_parity(x): the parity of the number of ones in the bit representation.
- gcd(x, y): the greatest common divisor of two numbers.
- __int128_t: the 128-bit integer type. Does not support input/output.

1.10 HashMap

```
// #include <bits/extc++.h>
// #include <ext/pb_ds/assoc_container.hpp>

struct splitmix64_hash {
    static uint64_t splitmix64(uint64_t x)
    {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) *
0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) *
0x94d049bb133111eb;
        return x ^ (x >> 31);
    }

    size_t operator()(uint64_t x) const
    {
        static const uint64_t
FIXED_RANDOM =
std::chrono::steady_clock::now().time_since
_epoch().count();
        return splitmix64(x +
FIXED_RANDOM);
    }
};

// template <typename K, typename V,
typename Hash = splitmix64_hash>
// using HashMap =
__gnu_pbds::gp_hash_table<K, V, Hash>;
// template <typename K, typename Hash =
splitmix64_hash>
// using HashSet = HashMap<K,
__gnu_pbds::null_type, Hash>;
typedef unordered_map<long long, int,
splitmix64_hash> safe_map;
```

2. Data Structures

2.1 Sparse Table

```
template <typename T, typename F>
class SparseTable {
    public:
        int n;
        vector<vector<T>> mat;
        F func;

        SparseTable(const vector<T>& a, const
F& f) : func(f) {
            n = static_cast<int>(a.size());
            int max_log = 32 - __builtin_clz(n);
            mat.resize(max_log);
            mat[0] = a;
            for (int j = 1; j < max_log; j++) {
                mat[j].resize(n - (1 << j) + 1);
                for (int i = 0; i <= n - (1 << j); i++)
{
                    mat[j][i] = func(mat[j - 1][i],
mat[j - 1][i + (1 << (j - 1))]);
                }
            }
        }

        T get(int from, int to) const {
            assert(0 <= from && from <= to &&
to <= n - 1);
            int lg = 32 - __builtin_clz(to - from +
1) - 1;
            return func(mat[lg][from], mat[lg][to
- (1 << lg) + 1]);
        }
};

auto fun = [&](int i,int j){return min(i,j);};
SparseTable<int, decltype(fun)>
sparse(arr,fun);
```

2.2 DSU

```
class DSU {
    public:
    vector<int> p;
    vector<int> sz;
    int n;
    DSU(int _n): n(_n) {
        p.resize(n);
```

```
        sz.assign(n, 1);
        iota(p.begin(), p.end(), 0);
    }
    int findRoot(int u) {
        if (p[u] == u) return u;
        return p[u] = findRoot(p[u]);
    }
    void combine(int u, int v) {
        int ru = findRoot(u);
        int rv = findRoot(v);
        if (ru == rv) return;
        if (sz[ru] < sz[rv]) swap(ru, rv);
        p[rv] = ru;
        sz[ru] += sz[rv];
    }
};
```

2.3 Simple Segment Tree

```
template <class Info>
struct SegTree {
    int n;
    vector<Info> info;
    SegTree() : n(0) {}
    SegTree(int n) : n(n), info(2 * n) {}
    SegTree(const vector<Info>& init) :
n(init.size()), info(2 * n) {
        for (int i = 0; i < n; i++) info[n + i] =
init[i];
        for (int i = n - 1; i > 0; i--) info[i] =
info[2 * i] + info[2 * i + 1];
    }

    void update(int i, Info val) {
        for (info[i += n] = val; i /= 2; )
            info[i] = info[2 * i] + info[2 * i + 1];
    }

    Info query(int l, int r) {
        Info resL, resR;
        for (l += n, r += n; l < r; l /= 2, r /= 2) {
            if (l & 1) resL = resL + info[l++];
            if (r & 1) resR = info[--r] + resR;
        }
        return resL + resR;
    }

    // Walk: Find first index i in [l, n) such that
check(query(l, i+1)) is true
    // Returns n if not found.
    template <class Predicate>
    int find_first(int l, Predicate check) {
        if (l >= n) return n;
        l += n;
        Info sum; // Identity

        do {
            while (l % 2 == 0) l >>= 1; // Move
up to align
            if (check(sum + info[l])) { // Answer
is in this subtree, drill down
                while (l < n) {
                    l <<= 1;
                    if (!check(sum + info[l])) {
                        sum = sum + info[l];
                        l++;
                    }
                }
                return l - n;
            }
            sum = sum + info[l];
            l++;
        } while ((l & -l) != l); // Stop if we cross
a power-of-2 boundary
        return n;
    }
};
struct Info {
    long long max_sub, max_pref, max_suff,
sum;
    Info() : max_sub(0), max_pref(0),
max_suff(0), sum(0) {}
    Info(int x) : max_sub(max(0, x)),
max_pref(max(0, x)), max_suff(max(0, x)),
sum(x) {}
};

Info operator+(const Info& a, const Info& b)
```

```
{
    Info res;
    res.sum = a.sum + b.sum;
    res.max_pref = max(a.max_pref, a.sum +
b.max_pref);
    res.max_suff = max(b.max_suff, b.sum +
a.max_suff);
    res.max_sub = max({a.max_sub,
b.max_sub, a.max_suff + b.max_pref});
    return res;
}

auto check = [&](Info current_prefix) {
return current_prefix.sum > K; };
```

2.4 Lazy Segment Tree

```
template <class Info, class Tag>
struct LazySegTree {
    int n;
    vector<Info> info;
    vector<Tag> lazy;

    LazySegTree() : n(0) {}
    LazySegTree(int n, Info _init = Info()) :
n(n), info(4 * n, _init), lazy(4 * n) {}
    LazySegTree(const vector<Info>& init) :
n(init.size()), info(4 * n), lazy(4 * n) {
        function<void(int, int, int)> build =
[&](int p, int l, int r) {
            if (l == r) {
                info[p] = init[l];
                return;
            }
            int m = (l + r) / 2;
            build(2 * p, l, m);
            build(2 * p + 1, m + 1, r);
            pull(p);
        };
        build(1, 0, n - 1);
    }

    void pull(int p) {
        info[p] = info[2 * p] + info[2 * p + 1];
    }

    void apply(int p, int l, int r, const Tag &v)
{
        info[p].apply(v, l, r);
        lazy[p].apply(v, l, r);
    }

    void push(int p, int l, int r) {
        int m = (l + r) / 2;
        apply(2 * p, l, m, lazy[p]);
        apply(2 * p + 1, m + 1, r, lazy[p]);
        lazy[p] = Tag();
    }

    void update(int p, int l, int r, int x, int y,
const Tag &v) {
        if (l > y || r < x) return;
        if (l >= x && r <= y) {
            apply(p, l, r, v);
            return;
        }
        push(p, l, r); // Pass range to push
        int m = (l + r) / 2;
        update(2 * p, l, m, x, y, v);
        update(2 * p + 1, m + 1, r, x, y, v);
        pull(p);
    }
    void update(int x, int y, const Tag &v) {
        update(1, 0, n - 1, x, y, v);
    }

    Info query(int p, int l, int r, int x, int y) {
        if (l > y || r < x) return Info();
        if (l >= x && r <= y) return info[p];
        push(p, l, r); // Pass range to push
        int m = (l + r) / 2;
        return query(2 * p, l, m, x, y) + query(2
* p + 1, m + 1, r, x, y);
    }
    Info query(int x, int y) {
        return query(1, 0, n - 1, x, y);
    }
};
```

```
struct Tag {
   long long add = 0;
   void apply(const Tag& t, int l, int r) {
      add += t.add;
   }
};

struct Info {
   long long sum = 0;
   void apply(const Tag& t, int l, int r) {
      sum += t.add * (r - l + 1);
   }
};
```

2.5 Fenwick Tree

```
struct Fenwick {
   int n;
   vector<long long> bit;
   Fenwick() {}
   Fenwick(int n) : n(n), bit(n + 1, 0) {}
   void add(int i, long long val) {
      for (; i <= n; i += i & -i) bit[i] += val;
   }
   long long sum(int i) const {
      long long s = 0;
      for (; i > 0; i -= i & -i) s += bit[i];
      return s;
   }
   long long sum(int l, int r) const {
      return sum(r) - sum(l - 1);
   }
};
```

2.6 2-D Fenwick

```
struct FT2 {
   vector<vi> ys; vector<FT> ft;
   FT2(int limx) : ys(limx) {}
   void fakeUpdate(int x, int y) {
      for (; x < sz(ys); x |= x + 1)
         ys[x].push_back(y);
   }
   void init() {
      for (vi& v : ys) sort(all(v)),
         ft.emplace_back(sz(v));
   }
   int ind(int x, int y) {
      return (int)(lower_bound(all(ys[x]), y) -
ys[x].begin());
   }
   void update(int x, int y, ll dif) {
      for (; x < sz(ys); x |= x + 1)
         ft[x].update(ind(x, y), dif);
   }
   ll query(int x, int y) {
      ll sum = 0;
      for (; x; x &= x - 1)
         sum += ft[x - 1].query(ind(x - 1, y));
      return sum;
   }
};
```

2.7 Trie

```
struct Node{
   Node* ch[2];
   int cnt[2];
   Node(){ ch[0]=ch[1]=0; cnt[0]=cnt[1]=0;
}
};

void insert(Node* t,int x){
   for(int i=31;i>=0;i--){
      int b=(x>>i)&1;
      if(!t->ch[b]) t->ch[b]=new Node();
      t->cnt[b]++;
      t=t->ch[b];
   }
}

void erase(Node* t,int x){
   for(int i=31;i>=0;i--){
      int b=(x>>i)&1;
      t->cnt[b]--;
      t=t->ch[b];
   }
}
int maxXor(Node* t,int x){
```

```
    int ans=0;
    for(int i=31;i>=0;i--){
        int b=(x>>i)&1;
        if(t->ch[b^1] && t->cnt[b^1]){
            ans|=(1<<i);
            t=t->ch[b^1];
        }else t=t->ch[b];
    }
    return ans;
}
```

2.8 Matrix

```
template <typename T>
vector<vector<T>> operator*(const
vector<vector<T>>& a, const
vector<vector<T>>& b)
{
    if (a.empty() || b.empty()) {
        return { {} };
    }
    vector<vector<T>> c(a.size(),
vector<T>(b[0].size()));
    for (int i = 0; i < static_cast<int>(c.size());
i++) {
        for (int j = 0; j <
static_cast<int>(c[0].size()); j++) {
            c[i][j] = 0;
            for (int k = 0; k <
static_cast<int>(b.size()); k++) {
                c[i][j] += a[i][k] * b[k][j];
            }
        }
    }
    return c;
}

template <typename T>
vector<vector<T>>&
operator*=(vector<vector<T>>& a, const
vector<vector<T>>& b)
{
    return a = a * b;
}

template <typename T, typename U>
vector<vector<T>> power(const
vector<vector<T>>& a, const U& b)
{
    assert(b >= 0);
    vector<U> binary;
    U bb = b;
    while (bb > 0) {
        binary.push_back(bb & 1);
        bb >>= 1;
    }
    vector<vector<T>> res(a.size(),
vector<T>(a.size()));
    for (int i = 0; i < static_cast<int>(a.size());
i++) {
        res[i][i] = 1;
    }
    for (int j = (int)binary.size() - 1; j >= 0; j--)
{
        res *= res;
        if (binary[j] == 1) {
            res *= a;
        }
    }
    return res;
}
```

2.9 Ordered Set

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;

template <typename T>
using ordered_set = tree<T, null_type,
less<T>, rb_tree_tag,
tree_order_statistics_node_update>;

/* USAGE:
    ordered_set<int> s;
    s.insert(1);
```

```
  s.insert(2);
  s.erase(1);

  FUNCTIONS:
  1. s.order_of_key(k) : Number of items
strictly smaller than k / order of k in s
  2. *s.find_by_order(k) : K-th element in
the set (0-based index).
*/
```

3. Maths

3.1 nCr

```
vector<Mint> fact, invFact;
void precompute(int n) {
        fact.resize(n + 1); invFact.resize(n +
1);
   fact[0] = 1;
   for (int i = 1; i <= n; i++) fact[i] = fact[i -
1] * i;
   invFact[n] = fact[n].inv();
   for (int i = n - 1; i >= 0; i--) invFact[i] =
invFact[i + 1] * (i + 1);
}

Mint nCr(int n, int r) {
   if (r < 0 || r > n) return 0;
   return fact[n] * invFact[r] * invFact[n - r];
}
```

3.2 Linear Sieve

```
vector<int> least = { 0, 1 };
vector<int> primes;
void RunLinearSieve(int n){
   least.assign(n + 1, 0);
   primes.clear();
   for (int i = 2; i <= n; i++) {
      if (least[i] == 0) {
         least[i] = i;
         primes.push_back(i);
      }
      for (int x : primes) {
         if (x > least[i] || i * x > n) {
            break;
         }
         least[i * x] = x;
      }
   }
}
```

3.3 Segmented Sieve

```
vector<int> segSieve(int L,int R){
   if(L<2) L=2;
   vector<char> ok(R-L+1,1);
   for(int p:primes){
      if(p*p>R) break;
      int s = max(p*p, ((L+p-1)/p)*p);
      for(int x=s;x<=R;x+=p) ok[x-L]=0;
   }
   vector<int> res;
   for(int i=0;i<=R-L;i++)
      if(ok[i]) res.push_back(L+i);
   return res;
}
RunLinearSieve(sqrt(MaxR));
auto primesInRange = segSieve(L,R);
```

3.4 Factorize

```
template <typename T>
vector<pair<T, int>> Factorize(T x){
vector<pair<T, int>> ret;
      while (x > 1) {
         if (!ret.empty() && ret.back().first ==
least[x]) {
            ret.back().second++;
         } else {
            ret.emplace_back(least[x], 1);
         }
         x /= least[x];
      }
      return ret;
}

template <typename T>
vector<T> BuildDivisorsFromFactors(const
```

```
vector<pair<T, int>>& factors)
{
    vector<T> divisors = { 1 };
    for (auto& p : factors) {
        int sz = (int)divisors.size();
        for (int i = 0; i < sz; i++) {
            T cur = divisors[i];
            for (int j = 0; j < p.second; j++) {
                cur *= p.first;
                divisors.push_back(cur);
            }
        }
    }
    sort(divisors.begin(), divisors.end());
    return divisors;
}
```

3.5 Euler Totient

```
long long phi(long long n){
    long long res = n;
    for(auto [p,c] : factorize(n))
        res -= res / p;
    return res;
}
const int LIM = 5000000;
int phi[LIM];
void calculatePhi() {
    rep(i,0,LIM) phi[i] = i&1 ? i : i/2;
    for (int i = 3; i < LIM; i += 2)
        if(phi[i] == i)
            for (int j = i; j < LIM; j += i)
                phi[j] -= phi[j] / i;
}
```

3.6 Mobius

```
int mobius[N + 5];
mobius[1] = -1;
for (int i = 1; i < N; i++) {
        if (mobius[i]) {
            mobius[i] = -mobius[i];
            for (int j = 2 * i; j < N; j += i) {
                mobius[j] += mobius[i];
            }
        }
}
```

3.7 Extended Euclidean

```
template <typename T>
T extgcd(T a, T b, T& x, T& y)
{
    if (a == 0) {
        x = 0;
        y = 1;
        return b;
    }
    T p = b / a;
    T g = extgcd(b - p * a, a, y, x);
    x -= p * y;
    return g;
}
```

3.8 Diophantine

```
bool diophantine(ll a,ll b,ll c,ll &x,ll &y){
    ll x0,y0;
    ll g = extgcd(abs(a),abs(b),x0,y0);
    if(c % g) return false;
    x = x0 * (c/g);
    y = y0 * (c/g);
    if(a<0) x = -x;
    if(b<0) y = -y;
    return true;
}
```

3.9 Summation Formulas

$\sum i = n(n+1)/2$

$\sum i^2 = n(n+1)(2n+1)/6$

$\sum i^3 = [n(n+1)/2]^2$

$\sum i^4 = n(n+1)(2n+1)(3n^2+3n-1)/30$

AP:

$\sum (a+(i-1)d) = n(2a+(n-1)d)/2$

GP:

$\sum a\, r^i = a(1-r^n)/(1-r),\ r\neq 1$

$\sum_{\infty} a\, r^i = a/(1-r),\ |r|<1$

Power:

$\sum_{i=a..b} c^i = (c^{b+1}-c^a)/(c-1)$

3.10 CRT

```
ll crt(ll a, ll m, ll b, ll n) {
    if (n > m) swap(a, b), swap(m, n);
    ll x,y;
    ll g = extgcd(m, n, x, y);
    assert((a - b) % g == 0); // else no
solution
    x = (b - a) % n * x % n / g * m + a;
    return x < 0 ? x + m*n/g : x;
}
```

3.11 AGP

∑(a+id)r^i (i=0..n−1) =
[a(1−r^n) + d(r − n r^n + (n−1) r^{n+1})] /
(1−r)^2
Special:
d=0 → GP
r=1 → AP
∞ AGP (|r|<1):
a/(1−r) + d r/(1−r)^2

3.12 Prime Check (Miller Rabin)

```
bool IsPrime(long long n) {
    if (n < 2) return false;
    std::vector<long long> bases = {2, 325,
9375, 28178, 450775, 9780504,
1795265022};
    for (long long x : {2, 3, 5, 7, 11, 13, 17,
19, 23, 29}) {
        if (n % x == 0) return n == x;
    }
    if (n < 31LL * 31) return true;
    long long d = n - 1, s = 0;
    while ((d & 1) == 0) d >>= 1, ++s;

    auto modmul = [&](long long a, long long
b, long long m) -> long long {
        return (__int128)a * b % m;
    };
    auto modpow = [&](long long a, long long
e, long long m) -> long long {
        long long r = 1;
        while (e) {
            if (e & 1) r = modmul(r, a, m);
            a = modmul(a, a, m);
            e >>= 1;
        }
        return r;
    };

    for (long long a : bases) {
        if (a % n == 0) continue;
        long long x = modpow(a, d, n);
        if (x == 1 || x == n - 1) continue;
        bool witness = true;
        for (int r = 1; r < s; ++r) {
            x = modmul(x, x, n);
            if (x == n - 1) {
                witness = false;
                break;
            }
        }
        if (witness) return false;
    }
    return true;
}
```

3.13 Pollard Rho

```
ull pollard(ull n) {
    ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    auto f = [&](ull x) { return modmul(x, x,
n) + i; };
    while (t++ % 40 || __gcd(prd, n) == 1) {
        if (x == y)
            x = ++i, y = f(x);
        if ((q = modmul(prd, max(x, y) - min(x,
y), n)))
            prd = q;
        x = f(x), y = f(f(y));
    }
    return __gcd(prd, n);
}
vector<ull> factor(ull n) {
    if (n == 1) return {};
    if (isPrime(n)) return {n};
    ull x = pollard(n);
    auto l = factor(x), r = factor(n / x);
    l.insert(l.end(), all(r));
```

```
    return l;
}
```

3.14 Catalan Number

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}$$

$$C_{n+1} = \frac{2(2n+1)}{n+2}C_n,\ C_{n+1} = \sum C_i C_{n-i}$$

- sub-diagonal monotone paths in an n×n grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with with n+ 1 leaves (0 or 2 children).
- ordered trees with n+ 1 vertices.
- ways a convex polygon with n+2 sides can be cut into triangles by connecting vertices with straight lines.
- permutations of [n] with no 3-term increasing subseq.

4. Geometry

4.1 Convex Hull

```
struct pt {
    double x, y;
    bool operator == (pt const& t) const {
        return x == t.x && y == t.y;
    }
};

int orientation(pt a, pt b, pt c) {
    double v =
a.x*(b.y-c.y)+b.x*(c.y-a.y)+c.x*(a.y-b.y);
    if (v < 0) return -1; // clockwise
    if (v > 0) return +1; // counter-clockwise
    return 0;
}

bool cw(pt a, pt b, pt c, bool
include_collinear) {
    int o = orientation(a, b, c);
    return o < 0 || (include_collinear && o ==
0);
}
bool collinear(pt a, pt b, pt c) { return
orientation(a, b, c) == 0; }

void convex_hull(vector<pt>& a, bool
include_collinear = false) {
    pt p0 = *min_element(a.begin(), a.end(),
[](pt a, pt b) {
        return make_pair(a.y, a.x) <
make_pair(b.y, b.x);
    });
    sort(a.begin(), a.end(), [&p0](const pt& a,
const pt& b) {
        int o = orientation(p0, a, b);
        if (o == 0)
            return (p0.x-a.x)*(p0.x-a.x) +
(p0.y-a.y)*(p0.y-a.y)
                < (p0.x-b.x)*(p0.x-b.x) +
(p0.y-b.y)*(p0.y-b.y);
        return o < 0;
    });
    if (include_collinear) {
        int i = (int)a.size()-1;
        while (i >= 0 && collinear(p0, a[i],
a.back())) i--;
        reverse(a.begin()+i+1, a.end());
    }

    vector<pt> st;
    for (int i = 0; i < (int)a.size(); i++) {
        while (st.size() > 1 &&
!cw(st[st.size()-2], st.back(), a[i],
include_collinear))
            st.pop_back();
        st.push_back(a[i]);
    }

    if (include_collinear == false && st.size()
== 2 && st[0] == st[1])
        st.pop_back();
```

```
    a = st;
}
```

4.2 Circle Line Intersection

Circle: center (cx, cy), radius r
Line: ax + by + c = 0

```
int circleLine(double a,double b,double c,
            double cx,double cy,double r,
            vector<P>& res){
    c += a*cx + b*cy;
    double den = a*a + b*b;
    double d2 = c*c / den;
    if(d2 > r*r + EPS) return 0;
    double x0 = -a*c / den;
    double y0 = -b*c / den;
    if(fabs(d2 - r*r) < EPS){
        res.push_back({x0+cx, y0+cy});
        return 1;
    }
    double h = sqrt(r*r - d2);
    double len = sqrt(den);
    double ux = -b/len, uy = a/len;

    res.push_back({x0 + ux*h + cx, y0 + uy*h
+ cy});
    res.push_back({x0 - ux*h + cx, y0 - uy*h
+ cy});
    return 2;
}
```

4.3 Circle Circle Intersection

Circle 1: (x1,y1), r1
Circle 2: (x2,y2), r2

```
int circleCircle(double x1,double y1,double
r1,double x2,double y2,double
r2,vector<P>& res){
    double dx = x2-x1, dy = y2-y1;
    double d = hypot(dx,dy);
    if(d > r1+r2+EPS) return 0;
    if(d < fabs(r1-r2)-EPS) return 0;
    if(d < EPS && fabs(r1-r2) < EPS) return
-1; // infinite
    double a = (r1*r1 - r2*r2 + d*d) / (2*d);
    double h2 = r1*r1 - a*a;
    if(h2 < 0) h2 = 0;
    double h = sqrt(h2);
    double xm = x1 + a*dx/d;
    double ym = y1 + a*dy/d;
    double rx = -dy * (h/d);
    double ry =  dx * (h/d);
    res.push_back({xm + rx, ym + ry});
    if(h > EPS)
        res.push_back({xm - rx, ym - ry});
    return res.size();
}
```

4.4 Polygon Area

```
ll polygonArea2(const vector<pair<ll,ll>>&
p) {
    ll s = 0;
    int n = p.size();
    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;
        s += p[i].first * p[j].second
          - p[i].second * p[j].first;
    }
    return llabs(s); // = 2 * area
}
ll area2 = polygonArea2(p);
if (area2 % 2 == 0) cout << area2 / 2;
else cout << area2 / 2 << ".5";
```

5. Graphs

5.1 LCA

```
const int MAXN = 200005;
vector<int> adj[MAXN];
vector<int> euler;      // nodes in Euler tour
vector<int> depth;      // depth corresponding
to euler
int first[MAXN];        // first occurrence of
node in euler
int n, q;
vector<vector<int>> st; // sparse table (stores
index in euler)
int LOG;
/* DFS to build Euler tour */
```

```
void dfs(int v, int p, int d) {
    first[v] = euler.size();
    euler.push_back(v);
    depth.push_back(d);

    for (int to : adj[v]) {
        if (to == p) continue;
        dfs(to, v, d + 1);
        euler.push_back(v);
        depth.push_back(d);
    }
}
/* Build Sparse Table for RMQ on depth */
void build_rmq() {
    int m = depth.size();
    LOG = 32 - __builtin_clz(m);
    st.assign(LOG, vector<int>(m));
    for (int i = 0; i < m; i++) st[0][i] = i;
    for (int k = 1; k < LOG; k++) {
        for (int i = 0; i + (1 << k) <= m; i++){
            int x = st[k-1][i];
            int y = st[k-1][i + (1 << (k-1))];
            st[k][i] = (depth[x] < depth[y] ? x :
y);
        }
    }
}
/* RMQ query on depth array (returns index
in euler) */
int rmq(int l, int r) {
    int len = r - l + 1;
    int k = 31 - __builtin_clz(len);
    int x = st[k][l];
    int y = st[k][r - (1 << k) + 1];
    return (depth[x] < depth[y] ? x : y);
}
/* LCA query */
int lca(int u, int v) {
    int l = first[u];
    int r = first[v];
    if (l > r) swap(l, r);
    int idx = rmq(l, r);
    return euler[idx];
}
```

5.2 SCC

```
vector<vector<int>> g, rg;
vector<int> ord, comp;
vector<char> vis;
void dfs1(int v){
    vis[v]=1;
    for(int u:g[v]) if(!vis[u]) dfs1(u);
    ord.push_back(v);
}
void dfs2(int v){
    vis[v]=1;
    comp.push_back(v);
    for(int u:rg[v]) if(!vis[u]) dfs2(u);
}
void kosaraju(int n){
    vis.assign(n,0);
    for(int i=0;i<n;i++) if(!vis[i]) dfs1(i);
    vis.assign(n,0);
    reverse(ord.begin(),ord.end());
    for(int v:ord) if(!vis[v]){
        dfs2(v);
        // comp = one SCC
        comp.clear();
    }
}
```

5.3 Bellman Ford

```
struct Edge{ int u,v,w; };
vector<Edge> e;
vector<long long> d;

void bellman(int n,int src){
    d.assign(n,INF);
    d[src]=0;
    for(int i=0;i<n-1;i++)
      for(auto &ed:e)
        if(d[ed.u]<INF)
          d[ed.v]=min(d[ed.v],d[ed.u]+ed.w);
}
```

5.4 Articulation Point

```
vector<vector<int>> g;
```

```
vector<int> tin, low;
vector<char> vis;
vector<int> isCut;
int timer;
void dfs(int v,int p){
    vis[v]=1;
    tin[v]=low[v]=timer++;
    int ch=0;
    for(int u:g[v]){
        if(u==p) continue;
        if(vis[u]){
            low[v]=min(low[v],tin[u]);
        }else{
            dfs(u,v);
            low[v]=min(low[v],low[u]);
            if(low[u]>=tin[v] && p!=-1)
                isCut[v]=1;
            ch++;
        }
    }
    if(p==-1 && ch>1) isCut[v]=1;
}
```

5.5 Bridge in Graph

```
vector<vector<int>> g;
vector<int> tin, low;
vector<char> vis;
vector<pair<int,int>> bridges
int timer;

void dfs(int v,int p){
    vis[v]=1;
    tin[v]=low[v]=timer++;
    for(int u:g[v]){
        if(u==p) continue;
        if(vis[u]){
            low[v]=min(low[v],tin[u]);
        }else{
            dfs(u,v);
            low[v]=min(low[v],low[u]);
            if(low[u] > tin[v])
                bridges.push_back({v,u});
        }
    }
}
```

5.6 Matrix exp on graph

Count paths:
G^k[i][j] = #paths (k edges)

Shortest paths:
min-plus mult
C[i][j]=min(A[i][p]+B[p][j])
G⊙k → shortest with exactly k edges
n small, k large

5.7 Dijkstra

```
vector<vector<pair<int,int>>> g;

void dijkstra(int s, vector<long long>& d){
    int n=g.size();
    d.assign(n,INF);
priority_queue<pair<ll,int>,vector<pair<ll,in
t>>,greater<>> pq;
    d[s]=0; pq.push({0,s});
    while(!pq.empty()){
        auto [dist,v]=pq.top(); pq.pop();
        if(dist!=d[v]) continue;
        for(auto [u,w]:g[v]) {
            if(d[v]+w<d[u]) {
                d[u]=d[v]+w;
                pq.push({d[u],u});
            }
        }
    }
}
```

5.8 Centroid Decomposition

```
vector<vector<int>> adj;
vector<int> subtree_size;
// min_dist[v] := the minimal distance
between v and a red node
vector<int> min_dist;
vector<bool> is_removed;
vector<vector<pair<int, int>>> ancestors;

int get_subtree_size(int node, int parent = -1)
{
```

```
	subtree_size[node] = 1;
	for (int child : adj[node]) {
		if (child == parent ||
is_removed[child]) { continue; }
		subtree_size[node] +=
get_subtree_size(child, node);
	}
	return subtree_size[node];
}

int get_centroid(int node, int tree_size, int
parent = -1) {
	for (int child : adj[node]) {
		if (child == parent ||
is_removed[child]) { continue; }
		if (subtree_size[child] * 2 >
tree_size) {
			return
get_centroid(child, tree_size, node);
		}
	}
	return node;
}

/**
 * Calculate the distance between current
`node` and the `centroid` it belongs
 * to. The distances between a node and all
its centroid ancestors are stored
 * in the vector `ancestors`.
 * @param cur_dist the distance between
`node` and `centroid`
 */
void get_dists(int node, int centroid, int
parent = -1, int cur_dist = 1) {
	for (int child : adj[node]) {
		if (child == parent ||
is_removed[child]) { continue; }
		cur_dist++;
		get_dists(child, centroid,
node, cur_dist);
		cur_dist--;
	}

ancestors[node].push_back({centroid,
cur_dist});
}

void build_centroid_decomp(int node = 0) {
	int centroid = get_centroid(node,
get_subtree_size(node));

	/*
	 * For all nodes in the subtree rooted
at `centroid`, calculate their
	 * distances to the centroid
	 */
	for (int child : adj[centroid]) {
		if (is_removed[child]) {
continue; }
		get_dists(child, centroid,
centroid);
	}

	is_removed[centroid] = true;
	for (int child : adj[centroid]) {
		if (is_removed[child]) {
continue; }
		// build the centroid
decomposition for all child components

build_centroid_decomp(child);
	}
}
```

5.9 Toposort

```
vi topoSort(const vector<vi>& gr) {
vi indeg(sz(gr)), q;
for (auto& li : gr) for (int x : li)
indeg[x]++;
rep(i,0,sz(gr)) if (indeg[i] == 0)
q.push_back(i);
rep(j,0,sz(q)) for (int x : gr[q[j]])
if (--indeg[x] == 0) q.push_back(x);
return q;
}
```

5.10 Find Cycles

```
// Returns list of cycles (each cycle is a list of vertices)
// adj: adjacency list
// bound_cnt/bound_sz: limits to prevent TLE on large outputs
vector<vector<int>> find_cycles(const vector<vector<int>> &adj, int bound_cnt = 1e9, int bound_sz = 1e9) {
    vector<vector<int>> cycles;
    vector<int> was(adj.size(), -1), st;
    int tot_sz = 0;

    auto dfs = [&](auto&& self, int u, int p) -> void {
        was[u] = st.size();
        st.push_back(u);
        for (int v : adj[u]) {
            if (v == p) continue;
            if (was[v] >= 0) {
                vector<int> cyc;
                for (int i = was[v]; i < (int)st.size(); i++) cyc.push_back(st[i]);
                cycles.push_back(cyc);
                tot_sz += cyc.size();
                if ((int)cycles.size() >= bound_cnt || tot_sz >= bound_sz) return;
            }
            else if (was[v] == -1) {
                self(self, v, u);
                if ((int)cycles.size() >= bound_cnt || tot_sz >= bound_sz) return;
            }
        }
        st.pop_back();
        was[u] = -2;
    };

    for (int i = 0; i < (int)adj.size(); i++) {
        if ((int)cycles.size() >= bound_cnt) break;
        if (was[i] == -1) dfs(dfs, i, -1);
    }
    return cycles;
}
```

5.11 2-Sat

```
const ll inf = LLONG_MAX;
struct Ed { int a, b, w, s() { return a < b ? a : -a; }};
struct Node { ll dist = inf; int prev = -1; };
void bellmanFord(vector<Node>& nodes, vector<Ed>& eds, int s) {
    nodes[s].dist = 0;
    sort(all(eds), [](Ed a, Ed b) { return a.s() < b.s(); });
    int lim = sz(nodes) / 2 + 2; // /3+100 with shuffled vertices
    rep(i, 0, lim) for (Ed ed : eds) {
        Node cur = nodes[ed.a], &dest = nodes[ed.b];
        if (abs(cur.dist) == inf) continue;
        ll d = cur.dist + ed.w;
        if (d < dest.dist) {
            dest.prev = ed.a;
            dest.dist = (i < lim - 1 ? d : -inf);
        }
    }
    rep(i, 0, lim) for (Ed e : eds) {
        if (nodes[e.a].dist == -inf)
            nodes[e.b].dist = -inf;
    }
}
```

5.12 Edge Coloring

```
vi edgeColoring(int N, vector<pii> eds) {
    vi cc(N + 1), ret(sz(eds)), fan(N), free(N), loc;
    for (pii e : eds)
        ++cc[e.first], ++cc[e.second];
    int u, v, ncols = *max_element(all(cc)) + 1;
    vector<vi> adj(N, vi(ncols, -1));
    for (pii e : eds) {
        tie(u, v) = e;
        fan[0] = v;
```

```
        loc.assign(ncols, 0);
        int at = u, end = u, d, c = free[u], ind =
0, i = 0;
        while (d = free[v], !loc[d] && (v =
adj[u][d]) != -1)
            loc[d] = ++ind, cc[ind] = d, fan[ind]
= v;
        cc[loc[d]] = c;
        for (int cd = d; at != -1; cd ^= c ^ d, at =
adj[at][cd])
            swap(adj[at][cd], adj[end = at][cd ^ c
^ d]);
        while (adj[fan[i]][d] != -1) {
            int left = fan[i], right = fan[++i], e =
cc[i];
            adj[u][e] = left;
            adj[left][e] = u;
            adj[right][e] = -1;
            free[right] = e;
        }
        adj[u][d] = fan[i];
        adj[fan[i]][d] = u;
        for (int y : {fan[0], u, end})
            for (int& z = free[y] = 0; adj[y][z] !=
-1; z++);
    }
    rep(i, 0, sz(eds))
        for (tie(u, v) = eds[i]; adj[u][ret[i]] !=
v;)
            ++ret[i];
    return ret;
}
```

5.13 Euler Walk

```
vi eulerWalk(vector<vector<pii>>& gr, int
nedges, int src = 0) {
    int n = sz(gr);
    vi D(n), its(n), eu(nedges), ret, s = {src};
    D[src]++; // allow Euler path, not just
cycle
    while (!s.empty()) {
        int x = s.back(), y, e, &it = its[x], end =
sz(gr[x]);
        if (it == end) {
            ret.push_back(x);
            s.pop_back();
            continue;
        }
        tie(y, e) = gr[x][it++];
        if (!eu[e]) {
            D[x]--; D[y]++;
            eu[e] = 1;
            s.push_back(y);
        }
    }
    for (int x : D)
        if (x < 0 || sz(ret) != nedges + 1)
            return {};
    return {ret.rbegin(), ret.rend()};
}
```

6. Flows

6.1 Hungarian

```
// cost: n x m (n <= m), 0-indexed
template<class T>
T hungarian(const vector<vector<T>>&
cost){
    const T INF = numeric_limits<T>::max();
    int n = cost.size(), m = cost[0].size();
    vector<T> u(n+1), v(m+1), dist(m+1);
    vector<int> p(m+1), way(m+1),
used(m+1);
    for(int i=1;i<=n;i++){
        p[0] = i;
        int j0 = 0;
        fill(dist.begin(), dist.end(), INF);
        fill(used.begin(), used.end(), 0);
        do{
            used[j0] = 1;
            int i0 = p[j0], j1 = 0;
            T delta = INF;
            for(int j=1;j<=m;j++) if(!used[j]){
                T cur = cost[i0-1][j-1] - u[i0] -
v[j];
                if(cur < dist[j]) dist[j] = cur, way[j]
```

```
= j0;
            if(dist[j] < delta) delta = dist[j], j1
= j;
        }
        for(int j=0;j<=m;j++){
            if(used[j]) u[p[j]] += delta, v[j] -=
delta;
            else dist[j] -= delta;
        }
        j0 = j1;
    } while(p[j0] != 0);
    for(int j1; j0; j0 = j1){
        j1 = way[j0];
        p[j0] = p[j1];
    }
  }
  return -v[0];   // minimum cost
}
```

6.2 Dinic

```
struct FlowEdge {
  int v, u;
  long long cap, flow = 0;
  FlowEdge(int v, int u, long long cap) :
v(v), u(u), cap(cap) {}
};
struct Dinic {
  const long long flow_inf = 1e18;
  vector<FlowEdge> edges;
  vector<vector<int>> adj;
  int n, m = 0;
  int s, t;
  vector<int> level, ptr;
  queue<int> q;

  Dinic(int n, int s, int t) : n(n), s(s), t(t) {
    adj.resize(n);
    level.resize(n);
    ptr.resize(n);
  }

  void add_edge(int v, int u, long long cap)
{
    edges.emplace_back(v, u, cap);
    edges.emplace_back(u, v, 0);
    adj[v].push_back(m);
    adj[u].push_back(m + 1);
    m += 2;
  }

  bool bfs() {
    while (!q.empty()) {
      int v = q.front();
      q.pop();
      for (int id : adj[v]) {
        if (edges[id].cap ==
edges[id].flow)
          continue;
        if (level[edges[id].u] != -1)
          continue;
        level[edges[id].u] = level[v] + 1;
        q.push(edges[id].u);
      }
    }
    return level[t] != -1;
  }

  long long dfs(int v, long long pushed) {
    if (pushed == 0)
      return 0;
    if (v == t)
      return pushed;
    for (int& cid = ptr[v]; cid <
(int)adj[v].size(); cid++) {
      int id = adj[v][cid];
      int u = edges[id].u;
      if (level[v] + 1 != level[u])
        continue;
      long long tr = dfs(u, min(pushed,
edges[id].cap - edges[id].flow));
      if (tr == 0)
        continue;
      edges[id].flow += tr;
      edges[id ^ 1].flow -= tr;
      return tr;
    }
    return 0;
```

```
    }

    long long flow() {
        long long f = 0;
        while (true) {
            fill(level.begin(), level.end(), -1);
            level[s] = 0;
            q.push(s);
            if (!bfs())
                break;
            fill(ptr.begin(), ptr.end(), 0);
            while (long long pushed = dfs(s,
flow_inf)) {
                f += pushed;
            }
        }
        return f;
    }
};
```

6.3 Push Relabel

```
const int inf = 1000000000;
int n;
vector<vector<int>> capacity, flow;
vector<int> height, excess;

void push(int u, int v) {
    int d = min(excess[u], capacity[u][v] -
flow[u][v]);
    flow[u][v] += d;
    flow[v][u] -= d;
    excess[u] -= d;
    excess[v] += d;
}

void relabel(int u) {
    int d = inf;
    for (int i = 0; i < n; i++) {
        if (capacity[u][i] - flow[u][i] > 0)
            d = min(d, height[i]);
    }
    if (d < inf)
        height[u] = d + 1;
}

vector<int> find_max_height_vertices(int s,
int t) {
    vector<int> max_height;
    for (int i = 0; i < n; i++) {
        if (i != s && i != t && excess[i] > 0) {
            if (!max_height.empty() && height[i]
> height[max_height[0]])
                max_height.clear();
            if (max_height.empty() || height[i] ==
height[max_height[0]])
                max_height.push_back(i);
        }
    }
    return max_height;
}

int max_flow(int s, int t)
{
    height.assign(n, 0);
    height[s] = n;
    flow.assign(n, vector<int>(n, 0));
    excess.assign(n, 0);
    excess[s] = inf;
    for (int i = 0; i < n; i++) {
        if (i != s)
            push(s, i);
    }

    vector<int> current;
    while (!(current =
find_max_height_vertices(s, t)).empty()) {
        for (int i : current) {
            bool pushed = false;
            for (int j = 0; j < n && excess[i]; j++)
{
                if (capacity[i][j] - flow[i][j] > 0
&& height[i] == height[j] + 1) {
                    push(i, j);
                    pushed = true;
                }
            }
            if (!pushed) {
```

```
            relabel(i);
            break;
        }
    }
  }
  return excess[t];
}
```

7. Strings

7.1 String Hashing

```
struct Hash{
  static const ll M1=1000000009, M2=1000000097;
  static const ll B1=91138233, B2=97266353;

  vector<ll> p1,p2;
  vector<pair<ll,ll>> h;

  Hash(const string& s){
    int n=s.size();
    p1.resize(n+1); p2.resize(n+1);
    h.resize(n+1);
    p1[0]=p2[0]=1;
    h[0]={0,0};
    for(int i=0;i<n;i++){
      p1[i+1]=p1[i]*B1%M1;
      p2[i+1]=p2[i]*B2%M2;
      h[i+1].first =(h[i].first *B1 + s[i])%M1;
      h[i+1].second=(h[i].second*B2 + s[i])%M2;
    }
  }
  pair<ll,ll> get(int l,int r){
    ll x1=(h[r+1].first - h[l].first*p1[r-l+1])%M1;
    ll x2=(h[r+1].second- h[l].second*p2[r-l+1])%M2;
    if(x1<0) x1+=M1;
    if(x2<0) x2+=M2;
    return {x1,x2};
  }
};
```

7.2 Longest Prefix

```
// pi[i] represents longest prefix which is also suffix for s[0....i]
vector<int> pi(const string& s){
  int n=s.size();
  vector<int> p(n);
  for(int i=1;i<n;i++){
    int j=p[i-1];
    while(j>0 && s[i]!=s[j]) j=p[j-1];
    if(s[i]==s[j]) j++;
    p[i]=j;
  }
  return p;
}
```

7.3 KMP

```
vector<int> findOccurrences(const string& p,const string& t){
  int n=p.size(), m=t.size();
  string s = p + "#" + t;
  auto pi = prefixFunction(s);
  vector<int> occ;
  for(int i=n+1;i<=n+m;i++)
    if(pi[i]==n)
      occ.push_back(i-2*n);
  return occ;
}
```

7.4 Manacher

```
vector<int> manacher(vector<int>& s) {
  int n = s.size();
  vector<int> t(2*n + 1);
  for(int i = 0; i < 2*n + 1; i++) {
    if(i % 2 == 0) t[i] = -1;
    else t[i] = s[i/2];
  }
  int m = t.size();
  vector<int> p(m);
  for(int i = 0, l = 0, r = -1; i < m; i++) {
    int k = (i > r) ? 1 : min(p[l + r - i], r - i +
```

```
1);
        while(i - k >= 0 && i + k < m && t[i -
k] == t[i + k]) k++;
        p[i] = k--;
        if(i + k > r) {
            l = i - k;
            r = i + k;
        }
    }
    return p;
}
```

7.5 String Trie

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int nxt[26];
    int pref = 0; // #strings that pass through
this node (including those ending deeper)
    int end = 0;  // #strings that end exactly
here
    Node(){ fill(nxt, nxt+26, -1); }
};

struct Trie {
    vector<Node> t;
    Trie(){ t.reserve(1<<16);
t.emplace_back(); } // root = 0

    void insert(const string &s){
        int v = 0;
        for(char ch : s){
            int c = ch - 'a';
            if(t[v].nxt[c] == -1){
                t[v].nxt[c] = (int)t.size();
                t.emplace_back();
            }
            v = t[v].nxt[c];
            t[v].pref++;
        }
        t[v].end++;
    }

    int countWordsEqualTo(const string &s)
const {
        int v = 0;
        for(char ch : s){
            int c = ch - 'a';
            if(t[v].nxt[c] == -1) return 0;
            v = t[v].nxt[c];
        }
        return t[v].end;
    }

    int countWordsStartingWith(const string
&p) const {
        int v = 0;
        for(char ch : p){
            int c = ch - 'a';
            if(t[v].nxt[c] == -1) return 0;
            v = t[v].nxt[c];
        }
        return t[v].pref;
    }

    // erase one occurrence if present; return
true if removed
    bool erase(const string &s){
        if(countWordsEqualTo(s) == 0) return
false; // do not create nodes here!
        int v = 0;
        for(char ch : s){
            int c = ch - 'a';
            v = t[v].nxt[c];
            t[v].pref--;
        }
        t[v].end--;
        return true;
    }
};
```

7.6 Z Transform

```
vector<int> z_function(string &txt,string
&pat){
    string s = pat + "$" + txt;
    int n = s.size();
```

```
    vector<int> z(n);
    z[0] = n;
    int l = 0, r = 0;
    for(int i = 1; i < n; i++){
        if(i <= r)
            z[i] = min(r - i + 1, z[i - l]);
        while(i + z[i] < n && s[z[i]] == s[i +
z[i]])
            z[i]++;
        if(i + z[i] - 1 > r){
            l = i;
            r = i + z[i] - 1;
        }
    }
    vector<int> ans;
    for (int i=0;i<n;i++) {
        if (z[i]==pat.size()) {
            ans.push_back(i-pat.size()-1);
        }
    }
    return ans;
}
```

7. Misc

7.1 LIS

```
template<typename T>
int lis(const vector<T>& a) {
  vector<T> u;
  for (const T& x : a) {
    auto it = lower_bound(u.begin(), u.end(),
x);
    if (it == u.end()) {
      u.push_back(x);
    } else {
      *it = x;
    }
  }
  return (int) u.size();
}
template<typename T>
vector<T> lis(const vector<T>& a) {
    vector<T> u;
    vector<int> p(a.size()), ind;
    for (int i = 0; i < a.size(); i++) {
        T x = a[i];
        auto it = lower_bound(u.begin(),
u.end(), x);
        int pos = it - u.begin();
        if (it == u.end()) {
            u.push_back(x);
            ind.push_back(i);
        } else {
            *it = x;
            ind[pos] = i;
        }
        p[i] = pos ? ind[pos - 1] : -1;
    }
    vector<T> res;
    if (ind.empty())  return res;
    for (int cur = ind.back(); cur != -1; cur =
p[cur])
        res.push_back(cur);
    reverse(res.begin(), res.end());
    return res;
}
```

7.2 Mo's Algo

```
struct Query {
    int l, r, id;
};
int block;
vector<int> a;
map<int,int> freq;
long long cur;
vector<long long> ans;
void add(int i) {
    int x = a[i];
    if (freq[x] == 0) cur++;
    freq[x]++;
}
void rem(int i) {
    int x = a[i];
    freq[x]--;
    if (freq[x] == 0) cur--;
}
```

```
bool cmp(const Query &A, const Query &B)
{
    if (A.l / block != B.l / block)
        return A.l / block < B.l / block;
    if ((A.l / block) & 1)
        return A.r > B.r;
    return A.r < B.r;
}
void mos_algo(int n, vector<Query> &q) {
    block = max(1LL, (int)sqrt(n));
    sort(q.begin(), q.end(), cmp);
    int L = 0, R = -1;
    for (auto &qr : q) {
        while (L > qr.l) add(--L);
        while (R < qr.r) add(++R);
        while (L < qr.l) rem(L++);
        while (R > qr.r) rem(R--);
        ans[qr.id] = cur;
    }
}
```

7.3 Ordered Set

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;

template <typename T>
using ordered_set = tree<T, null_type,
less<T>, rb_tree_tag,
tree_order_statistics_node_update>;

/* USAGE:
  ordered_set<int> s;
  s.insert(1);
  s.insert(2);
  s.erase(1);

  FUNCTIONS:
  1. s.order_of_key(k) : Number of items
strictly smaller than k / order of k in s
  2. *s.find_by_order(k) : K-th element in
the set (0-based index).
*/
```

7.4 Fast Knapsack

```
int fast_knapsack(vector<int>& w, int t) {
    int n = w.size();
    int a = 0, b = 0;
    while (b < n && a + w[b] <= t) a +=
w[b++];

    if (b == n) return a;

    int m = *max_element(w.begin(),
w.end());

    vector<int> dp(2*m + 1, -1), old;

    dp[a + m - t] = b;

    for (int i = b; i < n; i++) {
        old = dp;
        for (int x = 0; x + w[i] <= 2*m; x++)
            if (old[x] >= 0)
                dp[x + w[i]] = max(dp[x + w[i]],
old[x]);

        for (int x = 2*m; x > m; x--)
            if (old[x] >= 0)
                dp[x] = min(dp[x], old[x]);
    }

    for (int s = t; s >= 0; s--)
        if (dp[s + m - t] >= 0)
            return s;

    return 0;
}
```

7.5 Ternary Search

```
template<class F>
int ternSearch(int a, int b, F f) {
    assert(a <= b);
    while (b - a >= 5) {
        int mid = (a + b) / 2;
```

```
        if (f(mid) < f(mid + 1)) a = mid; // (A)
        else b = mid + 1;
    }
    rep(i, a + 1, b + 1)
        if (f(a) < f(i)) a = i;          // (B)
    return a;
}
```

7.6 NTT

```
const int MOD = 998244353;
const int G = 3;
ll modpow(ll a, ll e) {
    ll r = 1;
    while (e) {
        if (e & 1) r = r * a % MOD;
        a = a * a % MOD;
        e >>= 1;
    }
    return r;
}
void ntt(vector<int>& a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j |= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        ll wlen = modpow(G, (MOD - 1) / len);
        if (invert) wlen = modpow(wlen, MOD
- 2);
        for (int i = 0; i < n; i += len) {
            ll w = 1;
            for (int j = 0; j < len / 2; j++) {
                int u = a[i + j];
                int v = (int)(w * a[i + j + len / 2] %
MOD);
                a[i + j] = u + v < MOD ? u + v : u
+ v - MOD;
                a[i + j + len / 2] = u - v >= 0 ? u - v
: u - v + MOD;
                w = w * wlen % MOD;
            }
        }
    }
    if (invert) {
        ll inv_n = modpow(n, MOD - 2);
        for (int &x : a) x = x * inv_n % MOD;
    }
}
// Multiply polynomials a and b, keep
degrees < max_deg
vector<int> convolution(vector<int> a,
vector<int> b, int max_deg) {
    int need = min((int)a.size() + (int)b.size() -
1, max_deg);
    int n = 1;
    while (n < need) n <<= 1;
    a.resize(n);
    b.resize(n);
    ntt(a, false);
    ntt(b, false);
    for (int i = 0; i < n; i++)
        a[i] = (ll)a[i] * b[i] % MOD;
    ntt(a, true);
    a.resize(need);
    return a;
}
```

7.7 Some other STL functions/containers

```
 bool found = binary_search(all(v), x);
// removes duplicates for vector v
sort(all(v)); v.erase(unique(all(v)), v.end());
// list data structure [next(it), prev(it) works]
list<int> l; l.push_back(10);  l.push_front(5);
// Move iterator forward (O(K) cost)
advance(it, 2); [else use it++, it–;]
// Insert 99 BEFORE 'it',l.insert(it, 99);
// erasing elements
l.pop_back();  l.pop_front();
// Returns iterator to the element AFTER the
erased one,   it = l.erase(it);  [O(1)]
//other l.sort();l.clear();l.reverse(); l.unique();
l.sort(greater<int>()) [for unique sort first]
//Move all l2 elements to l before iterator pos
O(1),  l.splice(pos,l2);
```